

Experiment – LAB MST

Student Name: Mimanshu Gahlaut

UID: 24BAI70038

Branch: CSE - AIML

Section/Group: 24AIT_KRG-1_B

Semester: 5

Date of Performance: 28/9/26

Subject Name: ADBMS

Subject Code: 24CSP-310

(A)

A company stores employees and their departments. Display employees whose salary is greater than the average salary of their own department, but only from departments having at least 3 employees.

Display Department Name, Employee Name, and Salary.

Constraint: Only a solution using JOINS and SUBQUERIES is allowed. Window Functions are not allowed.

dept_id	dept_name
1	IT
2	Sales
3	HR

emp_id	emp_name	salary	dept_id
101	Aman	90000	1
102	Neha	70000	1
103	Raj	50000	1
104	Priya	80000	2
105	Karan	60000	2
106	Riya	40000	2
107	Mohit	75000	3
108	Simran	55000	3

Department	Employee	Salary
IT	Aman	90000
Sales	Priya	80000

Solution:

```
CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30)
);
```

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary NUMERIC(10,2),  
    dept_id INT REFERENCES department(dept_id)  
);
```

```
INSERT INTO department VALUES  
(1, 'IT'),  
(2, 'Sales'),  
(3, 'HR');
```

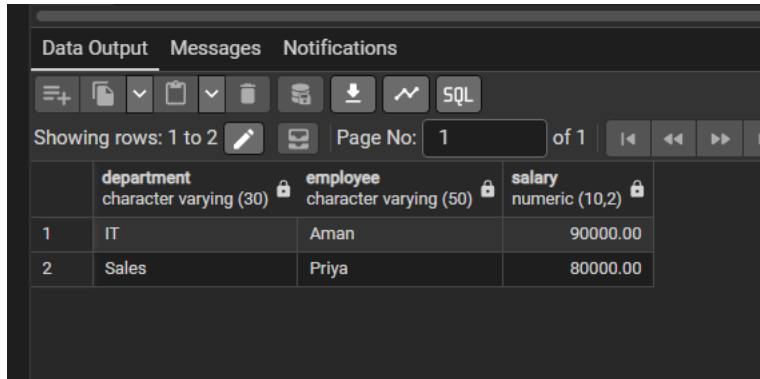
```
INSERT INTO employee VALUES  
(101, 'Aman', 90000, 1),  
(102, 'Neha', 70000, 1),  
(103, 'Raj', 50000, 1),  
(104, 'Priya', 80000, 2),  
(105, 'Karan', 60000, 2),  
(106, 'Riya', 40000, 2),  
(107, 'Mohit', 75000, 3),  
(108, 'Simran', 55000, 3);
```

```
SELECT  
    D.DEPT_NAME AS DEPARTMENT,  
    E.EMP_NAME AS EMPLOYEE,  
    E.SALARY AS SALARY  
FROM EMPLOYEE E  
JOIN DEPARTMENT D  
ON E.DEPT_ID = D.DEPT_ID  
WHERE E.SALARY > (  
    SELECT AVG(E1.SALARY)  
    FROM EMPLOYEE E1  
    WHERE E.DEPT_ID = E1.DEPT_ID  
)  
AND E.DEPT_ID IN (
```

```

SELECT DEPT_ID
FROM EMPLOYEE
GROUP BY DEPT_ID
HAVING COUNT (*) >= 3
);

```



	department character varying (30)	employee character varying (50)	salary numeric (10,2)
1	IT	Aman	90000.00
2	Sales	Priya	80000.00

(B)

A bank maintains customer information in bank_customer. Create a PostgreSQL trigger that automatically stores a log in customer_audit whenever a customer is inserted or deleted. Store Customer ID, Customer Name, action (ADDED/REMOVED), and current timestamp.

customer_id	customer_name	balance
1	Rahul	50000
2	Neha	75000

audit_id	customer_id	customer_name	action	action_time
1	3	Aman	ADDED	Current Time
2	1	Rahul	REMOVED	Current Time

Solution:

```

CREATE TABLE bank_customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    balance NUMERIC(10,2)
);

```

```

CREATE TABLE customer_audit (
    audit_id SERIAL PRIMARY KEY,
    customer_id INT,
    customer_name VARCHAR(50),
    action VARCHAR(20),
    action_time TIMESTAMP
);

```

```
);
```

```
INSERT INTO bank_customer VALUES
(1, 'Rahul', 50000),
(2, 'Neha', 75000);
```

```
CREATE OR REPLACE FUNCTION CUSTOMER_AUDIT_FUNCTION()
RETURNS TRIGGER AS $$
BEGIN
IF TG_OP = 'INSERT' THEN
    INSERT INTO CUSTOMER_AUDIT(CUSTOMER_ID, CUSTOMER_NAME, ACTION,
ACTION_TIME)
        VALUES (NEW.CUSTOMER_ID, NEW.CUSTOMER_NAME, 'ADDED',
CURRENT_TIMESTAMP);
    RETURN NEW;
ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO CUSTOMER_AUDIT(CUSTOMER_ID, CUSTOMER_NAME, ACTION,
ACTION_TIME)
        VALUES (OLD.CUSTOMER_ID, OLD.CUSTOMER_NAME, 'REMOVED',
CURRENT_TIMESTAMP);
    RETURN OLD;
END IF;
END;
$$ LANGUAGE PLPGSQL;
```

```
CREATE TRIGGER CUSTOMER_AUDIT
AFTER INSERT OR DELETE
ON BANK_CUSTOMER
FOR EACH ROW
EXECUTE FUNCTION CUSTOMER_AUDIT_FUNCTION();
```

```
INSERT INTO BANK_CUSTOMER
VALUES(3, 'AMAN', 60000);
```

```
SELECT * FROM CUSTOMER_AUDIT;
```

```
DELETE FROM BANK_CUSTOMER
WHERE CUSTOMER_ID = 1;
```

```
SELECT * FROM CUSTOMER_AUDIT;
```

Data Output Messages Notifications					
audit_id [PK] integer	customer_id integer	customer_name character varying (50)	action character varying (20)	action_time timestamp without time zone	
1	1	3 AMAN	ADDED	2026-09-28 15:22:59.486707	
2	2	1 Rahul	REMOVED	2026-09-28 15:24:22.421092	